[Lecture Notebook] Repetition Structures

MGS3001 Python Programming for Business, Spring 2025

Chad (Chungil Chae)

Thu, 13 March 2025

code_04_01_commission.py

```
# code_04_01_commission.py
# This program calculates sales commissions. 2
# Create a variable to control the loop.
keep_going = 'y'

# Calculate a series of commissions.
while keep_going == 'y':
    # Get a salesperson's sales and commission rate.
    sales = float(input('Enter the amount of sales: '))
    comm_rate = float(input('Enter the commission rate: '))

    # Calculate the commission.
    commission = sales * comm_rate

    # Display the commission.
    print('The commission is $',
    format(commission, ',.2f'), sep='')

# See if the user wants to do another one.
    keep_going = input('Do you want to calculate another ' + 'commission (Enter y for yes): ')
```

code_04_02_temperature.py

```
# code_04_02_temperature.py
# This program assists a technician in the process
```

```
# of checking a substance's temperature.

# Named constant to represent the maximum
# temperature.
MAX_TEMP = 102.5

# Get the substance's temperature.
temperature = float(input("Enter the substance's Celsius temperature: "))

# As long as necessary, instruct the user to
# adjust the thermostat.
while temperature > MAX_TEMP:
    print('The temperature is too high.')
    print('Turn the thermostat down and wait')
    print('5 minutes. Then take the temperature')
    print('again and enter it.')
    temperature = float(input('Enter the new Celsius temperature: '))

# Remind the user to check the temperature again
# in 15 minutes.
print('The temperature is acceptable.')
print('Check it again in 15 minutes.')
```

7 **code_04_03_infinite.py**

```
# code_04_03_infinite.py
# This program demonstrates an infinite loop.
# Create a variable to control the loop.
keep_going = 'y'

# Warning! Infinite loop!
while keep_going == 'y':
    # Get a salesperson's sales and commission rate.
    sales = float(input('Enter the amount of sales: '))
    comm_rate = float(input('Enter the commission rate: '))

    # Calculate the commission.
    commission = sales * comm_rate
    # Display the commission.
    print('The commission is $', format(commission, ',.2f'), sep='')
```


8 ## code_04_04_simple_loop1.py

```
# code_04_04_simple_loop1.py
# This program demonstrates a simple for loop
# that uses a list of numbers.

print('I will display the numbers 1 through 5.')
for num in [1, 2, 3, 4, 5]:
    print(num)
```

9 ## code_04_05_simple_loop2.py

```
# code_04_05_simple_loop2.py
print('I will display the odd numbers 1 through 9.')
for num in [1, 3, 5, 7, 9]:
    print(num)
```

10 ## code_04_06_simple_loop3.py

```
# code_04_06_simple_loop3.py
# This program also demonstrates a simple for
# loop that uses a list of strings.

for name in ['Winken', 'Blinken', 'Nod']:
    print(name)
```

11 ## code_04_07_simple_loop4.py

```
# code_04_07_simple_loop4.py
# This program demonstrates how the range
# function can be used with a for loop.

# Print a message five times.
for x in range(6):
    print('Hello world')
```



```
for num in range(1, 6):
    print(num)

for num in range(1, 10, 2):
    print(num)
```

12 **code_04_08_squares.py**

```
# code_04_08_squares.py
# This program uses a loop to display a
# table showing the numbers 1 through 10
# and their squares.

# Print the table headings.
print('Number\tSquare')
print('--------------')

# Print the numbers 1 through 10
# and their squares.
for number in range(1, 11):
    square = number**2
    print(number, '\t', square)
```

13 **code_04_09_speed_converter.py**

```
# code_04_09_speed_converter.py
# This program converts the speeds 60 kph
# through 130 kph (in 10 kph increments)
 # to mph.

START_SPEED = 60
END_SPEED = 131
INCREMENT = 10
CONVERSION_FACTOR = 0.6214 # Conversion factor

# Print the table headings.
print('KPH\tMPH')
print('--------------')
```



```
# Print the speeds.
for kph in range(START_SPEED, END_SPEED, INCREMENT):
    mph = kph * CONVERSION_FACTOR
    print(kph, '\t', format(mph, '.1f'))
```

14 **code_04_10_user_squares1.py**

```
# code_04_10_user_squares1.py
# This program uses a loop to display a
# table of numbers and their squares.

# Get the ending limit.
print('This program displays a list of numbers')
print('(starting at 1) and their squares.')
end = int(input('How high should I go? '))

# Print the table headings.
print()
print('Number\tSquare')
print('--------------')

# Print the numbers and their squares.
for number in range(1, end + 1):
    square = number**2
    print(number, '\t', square)
```

15 **code_04_11_user_squares2.py**

```
# code_04_11_user_squares2.py
# This program uses a loop to display a
# table of numbers and their squares.

# Get the starting value.
print('This program displays a list of numbers')
print('and their squares.')
start = int(input('Enter the starting number: '))

# Get the ending limit.
end = int(input('How high should I go? '))
```



```
# Print the table headings.
print()
print('Number\tSquare')
print('--------------')

# Print the numbers and their squares.
for number in range(start, end + 1):
    square = number**2
    print(number, '\t', square)
```

16 **code_04_12_sum_numbers.py**

```
# code_04_12_sum_numbers.py
# This program calculates the sum of a series
# of numbers entered by the user.

MAX = 5 # The maximum number

# Initialize an accumulator variable.
total = 0.0

# Explain what we are doing.
print('This program calculates the sum of')
print(MAX, 'numbers you will enter.')

# Get the numbers and accumulate them.
for counter in range(MAX):
    number = int(input('Enter a number: '))
    total = total + number

# Display the total of the numbers.
print('The total is', total)
```

17 **code_04_13_property_tax.py**

```
# code_04_13_property_tax.py
# This program displays property taxes.

TAX_FACTOR = 0.0065 # Represents the tax factor.
```



```
# Get the first lot number.
print('Enter the property lot number')
print('or enter 0 to end.')
lot = int(input('Lot number: '))

# Continue processing as long as the user
# does not enter lot number 0.
while lot != 0:
    # Get the property value.
    value = float(input('Enter the property value: '))

    # Calculate the property's tax.
    tax = value * TAX_FACTOR

    # Display the tax.
    print('Property tax: $', format(tax, ',.2f'), sep='')

    # Get the next lot number.
    print('Enter the next lot number or')
    print('enter 0 to end.')
    lot = int(input('Lot number: '))
```

18 **code_04_14_gross_pay.py**

```
# code_04_14_gross_pay.py
# This program displays gross pay.
# Get the number of hours worked.
hours = int(input('Enter the hours worked this week: '))

# Get the hourly pay rate.
pay_rate = float(input('Enter the hourly pay rate: '))

# Calculate the gross pay.
gross_pay = hours * pay_rate

# Display the gross pay.
print('Gross pay: $', format(gross_pay, ',.2f'))
```


19 ## code_04_14_validation1.py

```
# code_04_14_validation1.py
# Get a test score.
score = int(input('Enter a test score: '))
# Make sure it is not less than 0.
while score < 0:
    print('ERROR: The score cannot be negative.')
    score = int(input('Enter the correct score: '))
```

20 ## code_04_14_validation2.py

```
# code_04_14_validation2.py
# Get a test score.
score = int(input('Enter a test score: '))
# Make sure it is not less than 0 or greater than 100.
while score < 0 or score > 100:
    print('ERROR: The score cannot be negative')
    print('or greater than 100.')
    score = int(input('Enter the correct score: '))
```

21 ## code_04_15_retail_no_validation.py

```
# code_04_15_retail_no_validation.py
# This program calculates retail prices.
MARK_UP = 2.5 # The markup percentage
another = 'y' # Variable to control the loop.

# Process one or more items.
while another == 'y' or another == 'Y':

    # Get the item's wholesale cost.
    wholesale = float(input("Enter the item's " + "wholesale cost: "))

    # Calculate the retail price.
    retail = wholesale * MARK_UP

    # Display the retail price.
```



```
    print('Retail price: $', format(retail, ',.2f'), sep='')

    # Do this again?
    another = input('Do you have another item? ' + '(Enter y for yes): ')
```

code_04_16_retail_with_validation.py

```
# code_04_16_retail_with_validation.py
# This program calculates retail prices.
MARK_UP = 2.5 # The markup percentage
another = 'y' # Variable to control the loop.

# Process one or more items.
while another == 'y' or another == 'Y':

    # Get the item's wholesale cost.
    wholesale = float(input("Enter the item's " + "wholesale cost: "))

    # Validate the wholesale cost.
    while wholesale < 0:
        print('ERROR: the cost cannot be negative.')
        wholesale = float(input('Enter the correct ' + 'wholesale cost: '))

    # Calculate the retail price.
    retail = wholesale * MARK_UP

    # Display the retail price.
    print('Retail price: $', format(retail, ',.2f'), sep='')

    # Do this again?
    another = input('Do you have another item? ' + '(Enter y for yes): ')
```

code_04_17_nested_loop.py

```
# code_04_17_nested_loop.py
for hours in range(24):
    for minutes in range(60):
        for seconds in range(60):
            print(hours, ':', minutes, ':', seconds)
```

code_04_17_test_score_averages.py

```
# code_04_17_test_score_averages.py
# This program averages test scores. It asks the user for the
# number of students and the number of test scores per student.

# Get the number of students.
num_students = int(input('How many students do you have? '))

# Get the number of test scores per student.
num_test_scores = int(input('How many test scores per student? '))

# Determine each student's average test score.
for student in range(num_students):

    # Initialize an accumulator for test scores.
    total = 0.0

    # Get a student's test scores.
    print('Student number', student + 1)
    print('-----------------')
    for test_num in range(num_test_scores):
        print('Test number', test_num + 1, end='')
        score = float(input(': '))
        # Add the score to the accumulator.
        total += score

    # Calculate the average test score for this student.
    average = total / num_test_scores

    # Display the average.
    print('The average for student number', student + 1, 'is:', average)
    print()
```

code_04_18_rectangluar_pattern.py

```
# code_04_18_rectangluar_pattern.py
# This program displays a rectangular pattern # of asterisks.
rows = int(input('How many rows? '))
cols = int(input('How many columns? '))
```



```
for r in range(rows):
    for c in range(cols):
        print('*', end='')
    print()
```

code_04_19_triangle_pattern.py

```
# code_04_19_triangle_pattern.py
# This program displays a triangle pattern.
BASE_SIZE = 8
for r in range(BASE_SIZE):
    for c in range(r + 1):
        print('*', end='')
    print()
```

code_04_20_stair_step_pattern.py

```
# code_04_20_stair_step_pattern.py
# This program displays a stair-step pattern.
NUM_STEPS = 6
for r in range(NUM_STEPS):
    for c in range(r):
        print(' ', end='')
    print('#')
```

code_04_20_turtle_octagon.py

```
# code_04_20_turtle_octagon.py
# Drawing rect
import turtle
for x in range(4):
    turtle.forward(100)
    turtle.right(90)

# Move turtle
turtle.penup()
```



```
turtle.goto(200, 0)
turtle.pendown()

# Drawing Octagon
for x in range(8):
    turtle.forward(100)
    turtle.right(45)


# Turtle done
turtle.done()
```

29 **code_04_21_concentric_circles.py**

```
# code_04_21_concentric_circles.py
# Concentric circles
import turtle

# Named constants
NUM_CIRCLES = 20
STARTING_RADIUS = 20
OFFSET = 10
ANIMATION_SPEED = 0

# Setup the turtle.
turtle.speed(ANIMATION_SPEED)
turtle.hideturtle()

# Set the radius of the first circle
radius = STARTING_RADIUS

# Draw the circles.
for count in range(NUM_CIRCLES):
    # Draw the circle.
    turtle.circle(radius)

    #Get the coordinates for the next circle.
    x = turtle.xcor()
    y = turtle.ycor() - OFFSET

    # Calculate the radius for the next circle.
    radius = radius + OFFSET
```



```
    # Position the turtle for the next circle.
    turtle.penup()
    turtle.goto(x, y)
    turtle.pendown()

turtle.done()
```

30 ## code_04_22_spiral_circles.py

```
# code_04_22_spiral_circles.py
# This program draws a design using repeated circles.
import turtle

# Named constants
NUM_CIRCLES = 36
RADIUS = 100
ANGLE = 10
ANIMATION_SPEED = 0

# Number of circles to draw # Radius of each circle
# Angle to turn
# Animation speed
# Set the animation
turtle.speed(ANIMATION_SPEED)

# Draw 36 circles, with the turtle tilted
# by 10 degrees after each circle is drawn.
for x in range(NUM_CIRCLES):
    turtle.circle(RADIUS)
    turtle.left(ANGLE)

turtle.done()
```

31 ## code_04_23_spiral_lines.py

```
# code_04_23_spiral_lines.py
# This program draws
import turtle
```



```
# Named constants
START_X = -200
START_Y = 0
NUM_LINES = 36
LINE_LENGTH = 400
ANGLE = 170
ANIMATION_SPEED = 0

# Move the turtle to its initial position.
turtle.hideturtle()
turtle.penup()
turtle.goto(START_X, START_Y)
turtle.pendown()

# Set the animation speed.
turtle.speed(ANIMATION_SPEED)

# Draw 36 lines, with the turtle tilted
# by 170 degrees after each line is drawn.
for x in range(NUM_LINES):
    turtle.forward(LINE_LENGTH)
    turtle.left(ANGLE)

turtle.done()
```

